
DJED SHU: A STABLECOIN PROTOCOL WITH TWO ORACLE PRICES

ZAHNENTFERNER
The Stable Order
zahnentferner@gmail.com

YOGESH AGRAWAL
AOSSIE – The Australian Open Source Software Innovation and Education
agrawalyogesh5047@gmail.com

Abstract

This paper describes Djed Shu, a new version of the Djed stablecoin protocol. Like previous versions, Djed Shu is an autonomous crypto-backed stablecoin protocol. Djed Shu differs from and improves upon previous versions of Djed by using two oracle prices. This makes it more resilient and robust against oracle manipulations and delays.

Keywords: blockchain, cryptocurrency, stablecoin

1 Introduction

In the narrowest sense, a *stablecoin* is a cryptocurrency that has its price pegged to a fiat currency (e.g. USD) and is fully backed by reserves denominated in the same fiat currency. More broadly, a *stablecoin* can be defined as a digital asset that has mechanisms to maintain a low deviation of its price from a target price [3].

This paper describes Djed Shu, a new version of the Djed stablecoin protocol [4, 5]. Like previous versions, Djed Shu is an autonomous crypto-backed stablecoin protocol. It behaves like a central bank that, using its reserves, buys and sells its currency (the stablecoin) in order to keep it pegged within a target range. It is autonomous in the sense that it acts according to transparent and well-defined rules, that are executed by smart contracts independent of external influences. And it is crypto-backed in the sense that the bank keeps a cryptocurrency as its reserve asset. Besides stablecoins, the protocol also trades reservecoins in order to capitalize itself and maintain a reserve ratio significantly greater than 100% to absorb shocks in the price of the volatile cryptocurrency. Djed Shu differs from and improves upon previous

versions of Djed by using two oracle prices. This makes it more resilient and robust against oracle manipulations and delays.

Djed Shu is defined in Section 2. The protocol is designed to be as intuitive and minimalistic as possible, while still being stable, safe and secure. An oracle converter, to allow Djed Shu to be used with oracles that provide only one price, is described in Section 3. Current implementations of Djed Shu are briefly discussed in Section 4.

2 The Djed Shu Protocol

The Djed protocol's main purpose is to maintain the price of internally issued stablecoins (SCs) stable relative to a target price. The target price can be, for instance, the price of USD or EUR, or any other currency, asset or index. To achieve this, Djed maintains a reserve of basecoins (BCs), which is another cryptocurrency, such as BTC or ETH. The protocol issues SCs to users in exchange for BCs, which are then stored in the reserve. The amount of BCs a user has to pay for one stablecoin is defined by the exchange rate between the BC and the target asset, which is provided by an oracle (e.g., if a user buys 1 SC pegged to USD, the user pays an amount of BCs worth 1 USD). Similarly, the SCs can be sold back to Djed in exchange for a corresponding amount of BCs according to the current exchange rate.

As the price of BCs can be volatile, the protocol aims to have abundant reserves to preserve the peg to the target price even if the price of BCs drops significantly relative to the target asset. Additional BCs are acquired to the reserve through issuance of reservecoins (RCs). Purchase of RCs is attractive for users looking for greater returns. Not only RC holders indirectly benefit from the trading fees that accumulate in the reserve, but they are also exposed to a leverage effect: *ceteris paribus*, when the price of BCs grows, the price of RCs grows even more. Furthermore, the lower the reserve ratio, the stronger this leverage effect. Thus, buying reservecoins is more attractive precisely when the reserve ratio is lower. As buying reservecoins increases the reserve ratio, this varying leverage effect serves as an important mechanism for the sustainability of the protocol. On the other hand, the leverage effect also brings greater risks: *ceteris paribus*, when the price of BCs fall, the price of RCs fall even more. Thus, RC holders take the risk of BC price depreciation.

Djed's state is a triple (R, N_{SC}, N_{RC}) where:

- R is the number of basecoins (BCs) in Djed's reserve,
- N_{SC} is the number of stablecoins (SCs) in circulation,
- N_{RC} is the number of reservecoins (RCs) in circulation.

The protocol defines 4 operations to interact with the users:

1. The user **Buys SCs** from Djed by sending an amount of BCs to Djed. Djed then stores the provided BCs in the reserve and mints new SCs to the user.
2. The user **Sells SCs** to Djed. Djed then burns the provided SCs and returns to the user an amount of BCs from the reserve.
3. The user **Buys RCs** from Djed by sending an amount of BCs to Djed. Djed then stores the provided BCs in the reserve and mints new RCs to the user.
4. The user **Sells RCs** to Djed. Djed then burns the provided RCs and returns to the user an amount of BCs from the reserve.

Djed Shu uses an oracle system with two prices to enhance stability and protect reserves during times of high price volatility. By using two prices, Djed Shu acknowledges the fact that there is always some uncertainty in the price of one unit of the peg currency in BCs.

The *target price* of a SC in BCs now depends on the operation being performed. For operations requiring conservative pricing (user buying SCs or selling RCs), the target price is $P_{SC}^t = X^\uparrow$. For operations allowing liberal pricing (user selling SCs or buying RCs), the target price is $P_{SC}^t = X^\downarrow$. Djed Shu always chooses the price that maximizes preservation of its reserve. In other words, Djed always ensures that users are buying coins for a higher price and selling coins for a lower price.

Because Djed Shu's reserve may be insufficient to buy back all stablecoins for the target price, Djed Shu sets the actual *price* P_{SC} of SCs according to the following equation, where the target price P_{SC}^t depends on the specific operation:

$$P_{SC} = \begin{cases} P_{SC}^t & \text{if } N_{SC} = 0 \\ \min(P_{SC}^t, \frac{R}{N_{SC}}) & \text{otherwise.} \end{cases} \quad (1)$$

The portion of Djed Shu's reserve that would need to be used to buy back all stablecoins is known as its *liabilities*. Since the protocol buys back stablecoins using the lower price, liabilities are calculated as:

$$L(N_{SC}) = N_{SC} P_{SC}^\downarrow \quad (2)$$

The *reserve ratio* is calculated as:

$$r(R, N_{SC}) = \frac{R}{L(N_{SC})} = \frac{R}{N_{SC} P_{SC}^\downarrow} \quad (3)$$

Djed Shu maintains safety by having a *minimum reserve ratio* r_{min} and disallowing users from buying SCs or selling back RCs if, after these actions, $r(R, N_{SC}) < r_{min}$. To prevent

dilution for the RC holders, Djed Shu also has a *maximum reserve ratio* r_{max} and disallows users from buying more RCs if, before or after the purchase, $r(R, N_{SC}) > r_{max}$; unless $N_{SC} < N_{SC}^*$, where N_{SC}^* is the *threshold number of stablecoins* parameter.

The reserve surplus is Djed Shu's *equity*. Since equity calculations depend on which SC price is used for valuation, we define:

$$E^\downarrow(R, N_{SC}) = R - N_{SC}P_{SC}^\uparrow \quad (4)$$

$$E^\uparrow(R, N_{SC}) = R - N_{SC}P_{SC}^\downarrow \quad (5)$$

Djed Shu's equity is shared equally among RC holders. We define both a *buying price* and a *selling price* for RCs:

$$P_{RC}^b(R, N_{SC}, N_{RC}) = \begin{cases} \max\left(\frac{E^\uparrow(R, N_{SC})}{N_{RC}}, P_{RC}^{min}\right) & \text{if } N_{RC} > 0 \\ P_{RC}^{init} & \text{otherwise} \end{cases} \quad (6)$$

$$P_{RC}^s(R, N_{SC}, N_{RC}) = \frac{E^\downarrow(R, N_{SC})}{N_{RC}} \quad (7)$$

where P_{RC}^{min} is a parameter of Djed Shu.

Djed Shu's operations are summarized in Table 1. And Djed Shu's 6 parameters (r_{min} , r_{max} , fee , N_{SC}^* , P_{RC}^{min} , P_{RC}^{init}) are assumed to be set to values that satisfy the following constraints: $r_{min} > 1 + fee$; $r_{max} \geq r_{min}$; $0 < fee \leq 1$; $N_{SC}^* > 0$; $P_{RC}^{min} > 0$; $P_{RC}^{init} > P_{RC}^{min}$.

Action	Price	User Sends	User Receives	Condition
Buy SCs	$X^\uparrow$	$n(1 + fee)P_{SC}^\uparrow$	n SCs	$r(R, N_{SC}) \geq r_{min}$
Sell SCs	$X^\downarrow$	n SCs	$n(1 - fee)P_{SC}^\downarrow$	
Buy RCs	$X^\downarrow$	$n(1 + fee)P_{RC}^b(R, N_{SC}, N_{RC})$	n RCs	$r(R, N_{SC}) \leq r_{max}$ or $N_{SC} < N_{SC}^*$
Sell RCs	$X^\uparrow$	n RCs	$n(1 - fee)P_{RC}^s(R, N_{SC}, N_{RC})$	$r(R, N_{SC}) \geq r_{min}$

Table 1: Djed Shu's User Actions

The six protocol parameters must be carefully chosen to balance security, efficiency, and usability. Below we briefly explain the rationale for each parameter:

Minimum Reserve Ratio (r_{min}): This parameter determines the safety buffer against basecoin price crashes, enforced after every stablecoin purchase or reservecoin sale.

Maximum Reserve Ratio (r_{max}): This prevents excessive dilution of reservecoin holders when reserves are high by blocking RC purchases above this threshold, unless the stablecoin supply is below N_{SC}^* .

Trading Fee (fee): This fee is retained in the reserve on every operation, balancing protection against oracle manipulation with user costs.

Stablecoin Threshold (N_{SC}^*): This allows RC purchases even at high reserve ratios when stablecoin supply is low, facilitating protocol bootstrapping. For example, with $N_{SC}^* = 1000$, RC purchases remain available when SC supply is below 1000, even if $r > r_{max}$. This parameter helps bootstrap the protocol but should be set low enough to prevent abuse.

Minimum RC Price (P_{RC}^{min}): This floor price prevents RC values from collapsing to zero in extreme scenarios by ensuring the RC buying price never falls below this value, maintaining some value for RC holders even during severe market conditions.

Initial RC Price (P_{RC}^{init}): This sets the starting price for reservecoins when the RC supply is zero, providing a clear initial valuation at protocol launch. It should be set higher than P_{RC}^{min} to allow room for price movement.

3 Oracle Converter

In order to allow Djed Shu to work with oracles that provide just one price value [1], without requiring a specialised oracle that natively supplies two prices, We describe here an on-chain converter that transforms any single-price oracle into a two-price oracle as required by Djed Shu. In this converter, these two prices correspond to the *maximum* and *minimum* observed prices over a fixed rolling window of $T = 24$ hours. It aggregates all observations falling within each hour into an arithmetic mean. Missing hours are conservatively filled using the most recently observed price to preserve completeness of the window.

3.1 Hourly Price Aggregation

Let $p(t)$ denote the raw price returned by an external oracle at time t . Time is discretized into hourly intervals indexed by

$$h(t) = \left\lfloor \frac{t}{1 \text{ hour}} \right\rfloor \bmod T,$$

so that each hour corresponds to one index in the set $\{0, 1, \dots, T - 1\}$.

For each hour i , the contract maintains a stored value P_i representing the average price observed during that hour. When a new oracle update occurs at time t , let $i = h(t)$ be the current hour index. If k_i updates have already occurred within this hour, and P_i is the previous average, the contract computes the updated average price as

$$P_i \leftarrow \frac{k_i \cdot P_i + p(t)}{k_i + 1}, \quad k_i \leftarrow k_i + 1.$$

This procedure ensures that each P_i captures a time-weighted, manipulation-resistant summary of all price updates during that hour.

When the current hour index changes from i_{prev} to i_{curr} , the contract overwrites all intermediate hourly buckets with the most recent raw price observation. Formally, let

$$\Delta = (i_{\text{curr}} - i_{\text{prev}}) \bmod T.$$

For every offset $d \in \{1, 2, \dots, \Delta - 1\}$, the corresponding hourly bucket is updated as

$$P_{(i_{\text{prev}} + d) \bmod T} \leftarrow p(t).$$

This mechanism ensures that each hourly bucket always represents a value valid for its respective hour within the rolling window, regardless of the frequency of oracle updates.

3.2 Computation of Min and Max Prices

At any point in time, the system defines the minimum and maximum exchange prices as

$$X^\downarrow = \min_{0 \leq i < T} P_i, \quad X^\uparrow = \max_{0 \leq i < T} P_i.$$

These values are tracked incrementally. If the overwritten buckets or the updated hourly average may affect the global minimum or maximum, the contract recomputes the corresponding quantity across all T buckets. Otherwise, it simply compares the latest hourly average $P_{h(t)}$ with the existing extrema and updates the extremum index if needed.

4 Implementations

Djed Shu has the following implementations:

- **Djed Shu in Solidity for EVM-Compatible Blockchains:** Djed Shu has been implemented in Solidity for EVM-Compatible blockchains, using the blockchain's native currency as the basecoin. Since most oracles in EVM-compatible chains provide only a single latest price, this implementation also includes an *oracle converter* (implemented in the `ShuOracleConverter` contract), which transforms any single-price oracle into a time-windowed min/max oracle system. The converter maintains a 24-hour sliding window of hourly price averages $\bar{P}_h$ for hour h , computed as $\bar{P}_h = \frac{1}{n_h} \sum_{i=1}^{n_h} P_{h,i}$, where $P_{h,i}$ is the i -th price reading in hour h and n_h is the number of updates in that hour. Given the 24-hour window of prices $\{\bar{P}_0, \bar{P}_1, \dots, \bar{P}_{23}\}$, the converter computes $X^\downarrow = \min_{h \in [0, 23]} \bar{P}_h$ and $X^\uparrow = \max_{h \in [0, 23]} \bar{P}_h$, ensuring that $X^\downarrow \leq X^\uparrow$ always holds. The source code of this implementation can be found in <https://github.com/DjedAlliance/Djed-Solidity>.

- **Zephyr:** Zephyr (<https://zephyrprotocol.com/>) is a fork of Monero (<https://www.getmonero.org/>) where Djed Shu has been implemented natively in the blockchain's ledger layer [6]. This results in a blockchain that combines the privacy features of Monero with the stability of Djed. In Zephyr, the oracle's two prices are the latest price and a moving average.

5 Discussion

Previous versions of Djed [4,5], which rely on oracles that provide a single price, are sensitive to oracle delays and price manipulations. The protection against such problems is done solely through the fee for the buy and sell operations. For instance, if the fee is 2%, then the bank can tolerate a discrepancy of up to approximately 2% between the price provided by the oracle and the real price. In case of a greater discrepancy, due to oracle delays or price manipulations, users can arbitrage against the Djed bank because, essentially, Djed would be selling stablecoins or reservecoins for less than they are worth elsewhere or buying stablecoins or reservecoins for more than they are worth elsewhere. This would lead to a reduction in Djed's reserve. Thus, to protect against such issues, previous versions of Djed needed high fees, which resulted in a large (and fixed!) spread between the effective buy and sell prices for stablecoins and reservecoins.

Djed Shu, on the other hand, thanks to using oracles with two prices, has a dynamic and variable spread. During times of high volatility, the two prices will diverge and the spread will be larger. During times of low volatility, the spread will be smaller. This allows Djed Shu to have lower base fees and provides robust protection against various oracle attacks. With an oracle where the two prices are the minimum and maximum in a period of time or the latest price and a moving average, for example, price manipulations would have to be sustained for a long period of time to affect the one price that would allow an attacker to profit from the price manipulation. This rules out so-called flash loan price manipulation attacks that occur in a short period of time.

The oracle converter has also been designed to prevent attacks. For example, gas-based DoS attacks are prevented through selective min/max recalculation: full array scans are only executed when the sliding window moves past current extreme indices. The converter also maintains state consistency through safe initialization (all array elements initialized to current price), bounded array access using modular arithmetic, and monotonic update counters ensuring proper hourly averaging.

We conjecture that all stability theorems proven for previous versions of Djed hold for Djed Shu as well, since the theorem statements and proofs just need to be adapted to use one of the two prices, depending on the case. But adapting these proofs remains for future work.

Another direction for future work is the combination of Djed Shu and the Gluon Stablecoin

Protocol [2]. Current versions of Gluon rely on a single-price oracle, but tackle the sensitivity to oracles by making the fees depend on recent transaction volume. A combination of both protocols could provide even greater robustness against oracle delays and price manipulations.

References

- [1] Lorenz Breidenbach, Christian Cachin, Benedict Chan, Alex Coventry, Steve Ellis, Ari Juels, Farinaz Koushanfar, Andrew Miller, Brendan Magauran, Daniel Moroz, et al. Chainlink 2.0: Next steps in the evolution of decentralized oracle networks. *Chainlink Labs*, 1:1–136, 2021.
- [2] Bruno Woltzenlogel Paleo, Luca D’Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025.
- [3] Bruno Woltzenlogel Paleo. *Stablecoin*, pages 1–5. Springer Berlin Heidelberg, Berlin, Heidelberg, 2019.
- [4] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. <https://eprint.iacr.org/2021/1069>.
- [5] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. <https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program>.
- [6] Zephyr Protocol. Zephyr protocol whitepaper. https://zephyrprotocol.com/docs/zephyr_whitepaper.pdf, 2023. Accessed: 2026.